

ARCHITECTURE & SPECIFICATION

PRODUCTION BLUEPRINT

.NET 10 + REACT NATIVE

LocalStore Hyperlocal Platform

Complete Technical & Functional Documentation for the LocalStore Hyperlocal Commerce Engine

Ecosystem: LocalShop.Api & UI/local-shop-app

Date: September 2026

Status: Active Development

1. Executive Summary & Product Vision

LocalStore is a full-stack hyperlocal e-commerce and quick-commerce solution engineered to bridge neighborhood brick-and-mortar stores (Kirana shops, bakeries, dairies) with nearby residential consumers. The system supports three primary user profiles in an integrated multi-tenant interface:



Consumer Experience

- Real-time neighborhood catalog browsing
- 15-minute quick delivery pledge
- Instant cart with real-time price calculations
- Order tracking & automated instant refund tracing
- Simulated UPI and card payment ledger



Merchant Management

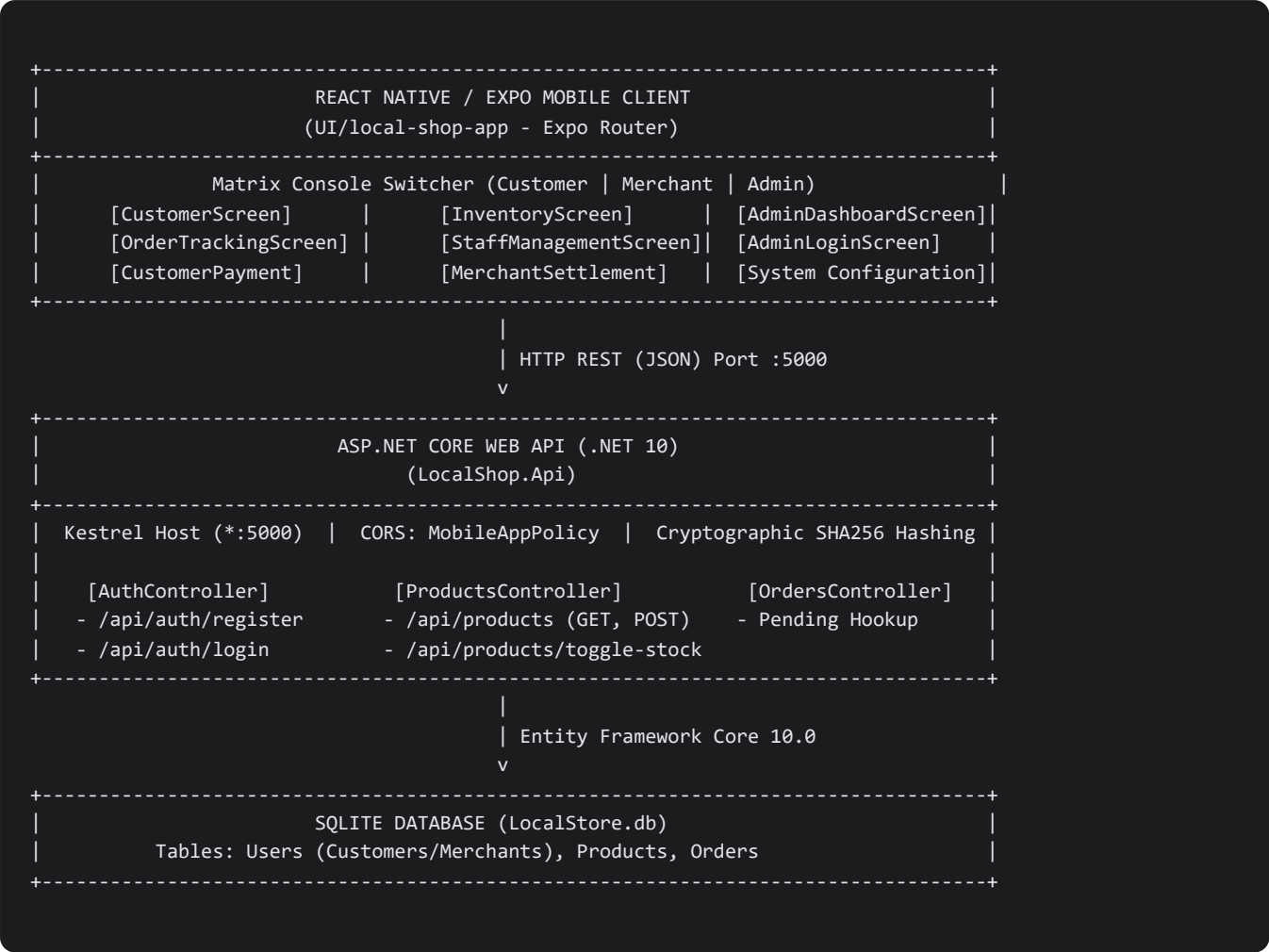
- Real-time inventory and stock control
- Direct-to-SQLite catalog ingestion
- Staff onboarding (Delivery, Billing) & authorization
- Weekly digital sales balance & bank payout settlement
- Store operational toggles & KYC licensing submission



Administrative Control

- Merchant onboarding audit & licensing review
- Customer account state directory
- Ecosystem GMV & commission financial reports
- Immutable system security audit logs
- Emergency maintenance mode & transaction fees

2. System Architecture & Technical Stack



Backend Specifications

- **Framework:** ASP.NET Core Web API (.NET 10.0 net10.0)
- **ORM:** Entity Framework Core 10 (SQLite Provider)
- **Database:** SQLite file-based DB (LocalStore.db)
- **Security:** SHA256 cryptographic password hashing
- **Network:** Binds to http://*:5000 with permissive CORS

Frontend Specifications

- **Framework:** React Native with Expo SDK 54 & Expo Router
- **Language:** TypeScript
- **Icons & Styling:** Lucide React Native, Native StyleSheet
- **Matrix Navigation:** Multi-role root switcher in HomeScreen
- **API Service:** Centralized BASE_URL config

3. Functional Modules & User Journeys

3.1 Customer Persona

FEATURE	SCREEN / COMPONENT	FUNCTIONAL CAPABILITY
Authentication	LoginScreen.tsx RegisterScreen.tsx	10-digit mobile number + password login. Registration records Full Name, Mobile, Delivery Address, and provisions a hashed user account in SQLite.
Storefront & Discovery	CustomerScreen.tsx	Shows active shop banner (Kirana Junction, Open status, 15-min delivery). Interactive search bar and category filter strip (All, Groceries, Dairy, Bakery) fetching live catalog items.
Wishlist & Cart	CustomerScreen.tsx	Favorites toggle with heart icons. Increment/decrement cart controls with stock validation. Sticky bottom checkout banner computing aggregate item count and total invoice.
Order Tracking & Refunds	OrderTrackingScreen.tsx	Displays active orders across lifecycle states (<i>Placed, Processing, Out for Delivery, Delivered, Cancelled</i>). One-tap instant cancellation triggers immediate automated refund state tracking.
Payment Ledger	CustomerPaymentScreen.tsx	Simulated UPI & Card payment flows connecting to .NET banking gateway. Chronological transaction ledger with status indicators (<i>Successful, Refunded</i>).
Account & Security	CustomerSettingsScreen.tsx	Manage primary delivery location, customer name, and configure recovery email address for password resets.

3.2 Merchant Persona

FEATURE	SCREEN / COMPONENT	FUNCTIONAL CAPABILITY
Merchant Onboarding	MerchantAuthScreen.tsx	Dedicated portal for store registration capturing Shop Name, Business Category, Mobile, and Password.
Inventory & Catalog	InventoryScreen.tsx	Direct-to-database ingestion form (Item Name, Price, Stock Units). Live catalog list with visual stock health indicators (< 5 units flagged with red warnings).
Staff Management	StaffManagementScreen.tsx	Authorize team members (Delivery Partners, Billing Clerks) with one-tap status toggling between <i>Active</i> and <i>Suspended</i> .
Settlements & Payouts	MerchantSettlementScreen.tsx	Withdrawable balance card calculating digital customer payments. One-tap payout dispatch to linked bank account (e.g., HDFC) with historical dispatch logs.
Operations & Compliance	MerchantSettingsScreen.tsx	Live Store Open/Closed operational switch, delivery radius limit (km), minimum invoice threshold (₹), and KYC Trade License submission for administrative audit.

3.3 Administrator Persona

FEATURE	SCREEN / COMPONENT	FUNCTIONAL CAPABILITY
Security Gate	AdminLoginScreen.tsx	Elevated administrative access challenge requiring master credentials (superadmin / admin123).
Merchant Approvals	AdminDashboardScreen.tsx	Review queue for new shop registrations and submitted trade licenses; execute immediate <i>Approve</i> or <i>Reject</i> decisions.
Customer Oversight	AdminDashboardScreen.tsx	Comprehensive customer directory detailing user identities, contact numbers, and account compliance status (<i>Active</i> vs <i>Flagged</i>).
Financial Reports	AdminDashboardScreen.tsx	Ecosystem metrics: Gross Merchandise Volume (GMV: ₹4,82,900), invoice volume (1,412), platform commissions (₹12,072.50), and total settled merchant capital.
Security Audit Logs	AdminDashboardScreen.tsx	Chronological event audit logging settlement transactions, unauthorized root access attempts, and origin IP actors.
System Governance	AdminDashboardScreen.tsx	Platform transaction commission fee configuration (%) and global emergency Infrastructure Maintenance Lock switch.

4. Backend API & Data Schemas

4.1 Database Entities (StoreDbContext)

ENTITY / TABLE	PRIMARY KEY	ATTRIBUTES & TYPES	BUSINESS CONTEXT
User	Id (TEXT, GUID)	Name (TEXT), Phone (TEXT, Unique), PasswordHash (TEXT, SHA256), Role (TEXT: Customer Merchant Admin), Address (TEXT)	Stores all authentication profiles, delivery addresses, and role identities.
Product	Id (TEXT, GUID)	Name (TEXT), Price (TEXT/Decimal), Category (TEXT), Stock (INTEGER), IsAvailable (INTEGER/Bool)	Central marketplace inventory catalog with availability and pricing.
Order	Id (TEXT, ORD-XXXXXX)	CustomerName (TEXT), ShopName (TEXT), ItemsCount (INTEGER), TotalAmount (TEXT/Decimal), Status (TEXT), RefundStatus (TEXT), PaymentMethod (TEXT)	Order lifecycle and fulfillment ledger supporting cancellation and refund tracking.

4.2 API Endpoint Inventory

HTTP METHOD	ENDPOINT ROUTE	CONTROLLER	PARAMETERS / BODY	OPERATIONAL STATUS
POST	/api/auth/register	AuthController	JSON: User (Name, Phone, PasswordHash, Address, Role)	LIVE & VERIFIED
POST	/api/auth/login	AuthController	JSON: LoginRequest (Phone, Password)	LIVE & VERIFIED
GET	/api/products	ProductsController	Query: ?category={cat}&search={text}	LIVE & VERIFIED
POST	/api/products	ProductsController	JSON: Product (Name, Price, Category, Stock, IsAvailable)	LIVE & VERIFIED
PUT	/api/products/toggle-stock/{id}	ProductsController	Path: id (Product GUID)	LIVE & VERIFIED
GET/POST	/api/orders/*	OrdersController	JSON / Path	PENDING CONTROLLER IMPLEMENTATION

5. Architectural Analysis & Next Roadmap Steps

Priority Item 1: Complete OrdersController.cs

The Entity Framework model and SQLite table for Orders are already scaffolded and migrated. However, OrdersController.cs is currently 0 bytes. Implementing endpoints for placing orders (POST /api/orders), fetching user order history (GET /api/orders), and cancellation (PUT /api/orders/{id}/cancel) will replace client mock arrays with persistent database records.

Priority Item 2: Itemized Order Details

Currently, the Order entity stores aggregated summaries (ItemCount, TotalAmount). Adding a relational OrderItem entity linking each order to specific products and quantities will enable itemized customer receipts and automatic inventory stock deduction upon order completion.

Priority Item 3: Industry-Standard JWT Token Authentication

Upgrading AuthController to issue signed JWT tokens will allow role-based authorization attributes ([Authorize(Roles = "Merchant")]) on sensitive administrative and inventory management routes.